信息与软件工程学院

编译技术

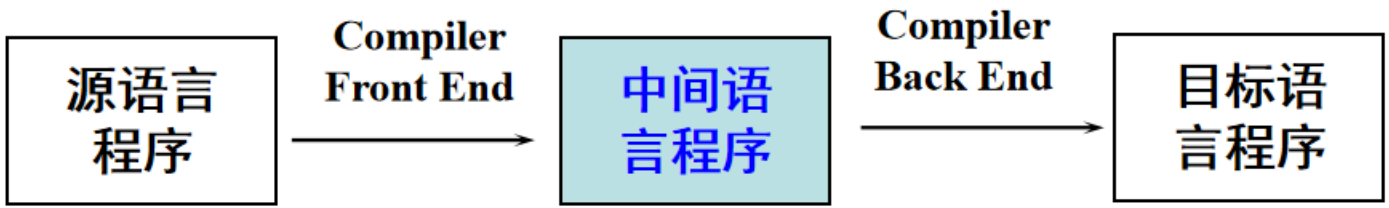
主讲教师：陈安龙

第5章 中间代码生成

- 中间代码简介
- 三元式
- 四元式
- 赋值表达式
- 控制流和布尔表达式的翻译

一、中间代码生成

中间代码生成是编译过程中的一个阶段，负责将语义分析的语法树或语法制导翻译结果转换成一种介于高级语言和机器语言之间的代码形式，称为中间代码。



使用中间语言的优点

① 独立于机器：中间代码与具体机器无关，便于移植。

② 简化优化：中间代码结构简单，便于进行各种优化。

③ 便于目标代码生成：中间代码是生成目标机器代码的基础。

二、中间代码的形式

① 三地址码

② 四元式

③ 三元式

④ 间接三元式

三地址码简介

三地址码是一种常用的中间代码形式，每条指令最多包含三个地址（操作数），

通常表示为：x：＝y op z

其中，x、y、z 可以是变量、常量或临时变量，op 是操作符。

三地址码结构简单，接近机器指令，便于优化和目标代码生成。

三地址码的种类

三地址码种类	格式示例	说明
赋值语句	x = y	简单赋值
算术运算语句	x = y op z	二元算术运算
单目运算语句	x = op y	单目运算，如：not,-
条件跳转语句	if x relop y goto L	条件跳转
无条件跳转语句	goto L	无条件跳转
标签语句	L:	跳转标签
函数调用语句	param x、call f,n	函数参数传递和调用
函数返回语句	return x	函数返回
数组元素访问语句	x = y[i]、x[i] = y	数组元素读写
结构体成员访问语句	x = y.field	结构体成员读写

三地址码示例

- 表达式 a = b * (-c) + b * (-c) 的三地址码：

(0) t1 = minus c

(1) t2 = b * t1

(2) t3= minus c

(3) t4 = b * t3

(4) t5 = t2 + t4

(5) a=t5

四元式

四元式表示通常表示为 (op, arg1, arg2, result)，其中 op 是操作符，arg1 和 arg2 是操作数，result 是结果。

例如: 表达式 a = b * (-c) + b * (-c)的四元式表示:

op	arg_1	arg_2	*result*
minus	c		t_1
*	b	t_1	t_2
minus	c		t_3
*	b	t_3	t_4
+	t_2	t_4	t_5
=	t_5		a

或者

(0) (-, c, , t1)
(1) (*, b, t1, t2)
(2) (-, c, , t3)
(3) (*, b, t3, t4)
(4) (+, t2, t4,t5)
(5) (=, t5, ,a)

三元式

三元式表示通常表示为 (op, arg1, arg2)，其中 op 是操作符，arg1 和 arg2 是操作数。结果用计算语句位置表示。

例如： 表达式 a = b * (-c) + b * (-c)三元式表示：

	op	arg_1	arg_2
0	minus	c	
1	*	b	(0)
2	minus	c	
3	*	b	(2)
4	+	(1)	(3)
5	=	a	(4)

或者

(0) (-, c,)
(1) (*, b, (0))
(2) (-, c,)
(3) (*, b, (2))
(4) (+, (1),(3))
(5) (=, a, (4))

间接三元式

间接三元式是一种中间代码表示形式，它使用一个间接寻址表来存储操作数和结果。

这种表示形式可以方便地进行各种优化，如公共子表达式消除、复制传播等。

例如：表达式 a = b * (-c) + b * (-c) 的间接三元式如下：

instuction	
35	(0)
36	(1)
37	(2)
38	(3)
39	(4)
40	(5)
	...

	op	arg_1	arg_2
0	minus	c	
1	*	b	(0)
2	minus	c	
3	*	b	(2)
4	+	(1)	(3)
5	=	a	(4)

静态单赋值形式

- 一种便于代码优化的中间表示
- 和三地址码的主要区别:所有赋值指令都是对不同名字的变量的赋值

三地址代码	静态单赋值形式
$p = a + b$	$p_1 = a + b$
$q = p - c$	$q_1 = p_1 - c$
$p = q * d$	$p_2 = q_1 * d$
$p = e - p$	$p_3 = e - p_2$
$q = p + q$	$q_2 = p_3 + q_1$

三、中间代码生成的基本方法

1. **递归遍历语法树**

   对抽象语法树进行递归遍历，生成对应的中间代码。

2. **语法制导翻译**

   ① 利用语法制导定义和语法制导翻译，在语法分析过程中插入生成中间代码的语义动作。

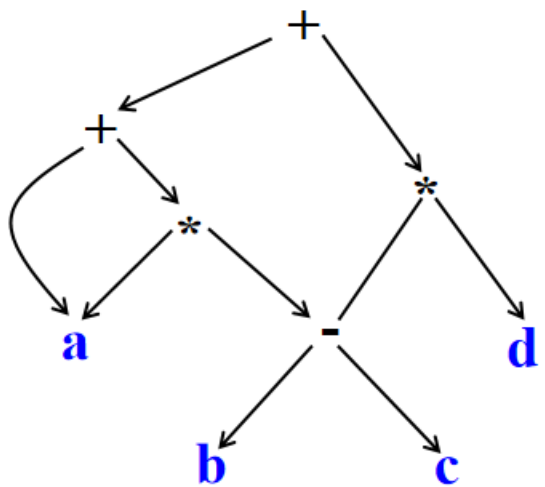
   ② 通过属性计算生成三地址码。

四、语法树的变体：有向无环图（DAG）

- 语法树（Parse Tree 或 Syntax Tree）是源程序语法结构的树形表示，反映了语法规则的推导过程。每个内部节点对应一个语法规则的产生式，叶子节点对应终结符。语法树结构清晰，但可能存在大量重复子树。
- 有向无环图（DAG）是一种图结构，具有方向且不包含环路。在编译中，DAG用来表示表达式的结构，允许共享公共子表达式，避免重复表示。
- 语法树中相同的子表达式会被重复表示，导致冗余。DAG通过合并相同的子表达式节点，实现共享，节省空间。便于进行公共子表达式消除等优化。

有向无环图（DAG）的示例

例如：表达式a+a*(b-c)+(b-c)*d的DAG图表示



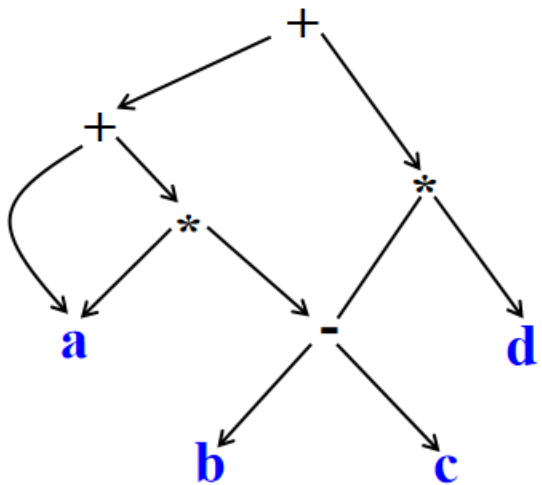
DAG

创建表达式的语法树的语法制导定义

产 生 式	语义规则
$E \rightarrow E_1+T$	{ E.*node*:=new *Node*('+',E_1.*node*,T.*node*) }
$E \rightarrow E_1-T$	{ E.*node*:=new *Node*('-',E_1.*node*,T.*node*) }
$E \rightarrow T$	{ E.*node*:= T.*node* }
$T \rightarrow T_1*F$	{ T.*node*:=new *Node*('*',T_1.*node*,F.*node*) }
$T \rightarrow (E)$	{ T.*node*:=E.*node* }
$T \rightarrow id$	{ T.*node*:=new *Leaf*(id,id.*entry*) }
$T \rightarrow num$	{ E.*node*:=new *Leaf*(num,num.*val*) }

创建表达式a+a*(b-c)+(b-c)*d的语法树DAG图

1) p_1 = *Leaf*(id,*entry-a*)
2) p_2 = *Leaf*(id,*entry-a*) = p_1
3) p_3 = *Leaf*(id,*entry-b*)
4) p_4 = *Leaf*(id,*entry-c*)
5) p_5 = *Node*('-',p_3,p_4)
6) p_6 = *Node*('*',p_1,p_5)
7) p_7 = *Node*('+',p_1,p_6)
8) p_8 = *Leaf*(id,*entry-b*) = p_3
9) p_9 = *Leaf*(id,*entry-c*) = p_4
10) p_{10} = *Node*('-',p_3,p_4) = p_5
11) p_{11} = *Leaf*(id,*entry-d*)
12) p_{12} = *Node*('*',p_5,p_{11})
13) p_{13} = *Node*('+',p_7,p_{12})

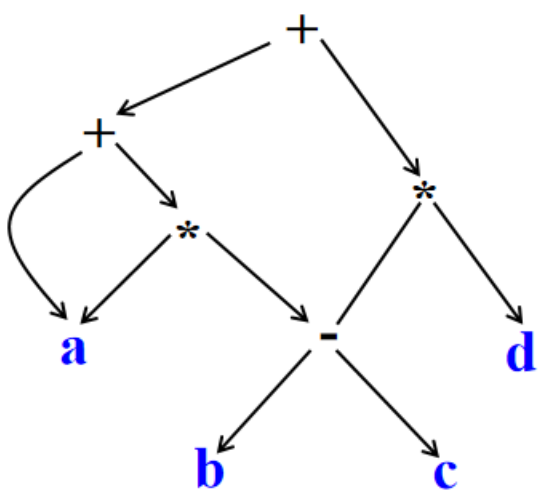


DAG

DAG图的存储结构

DAG图或语法树的结点一般存储在数组中，数组的每一行表示一个结点。

例如：表达式a+a*(b-c)+(b-c)*d图的数组存储



DAG

0	id	a在符号表的地址	
1	id	b在符号表的地址	
2	id	c在符号表的地址	
3	-	1	2
4	*	0	3
5	+	0	4
6	id	d在符号表的地址	
7	*	3	6
8	+	5	7

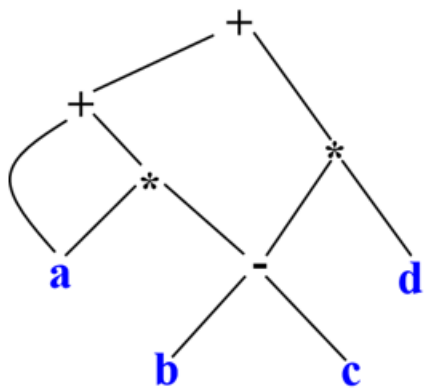
值编码，类似指针或者数组下标。

从DAG递归生成三地址码的算法（伪代码）

```
function generateCode(node) {
    if node.visited:
        return node.place          // 已生成代码，直接返回变量名
    if node is leaf:               // 变量或常量
        node.place = node.name
        node.visited = true
        return node.place
    left_place = generateCode(node.left)   // 递归生成左子节点代码
    right_place = generateCode(node.right) // 递归生成右子节点代码
    temp = newTemp()  // 分配临时变量
    // 生成三地址码
    emit(temp + " = " + left_place + " " + node.op + " " + right_place)
    node.place = temp
    node.visited = true
    return temp
}
```

递归生成三地址代码示例

- 将表达式 a+a*(b-c)+(b-c)*d翻译为如下三地址代码：



DAG

$t_1 = b - c;$

$t_2 = a * t_1;$

$t_3 = a + t_2;$

$t_4 = t_1 * d;$

$t_5 = t_3 + t_4;$

三地址代码

复习语法制导定义

① 法制导定义是在上下文无关文法的基础上，为每个文法符号定义属性，并为产生式定义语义规则。

② 属性分为综合属性（Synthesized Attributes）和继承属性（Inherited Attributes）。

③ 语义规则描述如何计算属性，通常用于实现语义分析和中间代码生成。

五、语法制导定义生成三地址代码的思路

① 将三地址代码作为属性值（通常是综合属性）附加到文法符号上。

② 在语法分析过程中，根据产生式的语义规则，逐步生成三地址代码。

③ 利用临时变量表示中间结果。

④ 通过属性传递，保证代码生成的顺序和正确性。

语法制导定义（翻译）的常用属性设计

① place存放表达式结果的变量名（临时变量或标识符）

② code表示生成的三地址代码序列。

③ next 用于控制流语句的下一条指令地址

④ newtemp是个函数，对它的调用将产生一个新的临时变量。

表达式生成三地址码的语法制导翻译

① S.code：语句对应的三地址代码序列

② E.place：表达式结果存放的变量名

③ E.code：表达式对应的三地址代码序列

④ newTemp()：生成新的临时变量名，如 t1, t2。

⑤ || 表示代码串连接。

⑥ id.name：表示标识符的名字。

```
S → id = E
    { print(E.code)
      print("id = " + E.place) }
E → E1 + E2
    { E.place = newTemp()
      E.code = E1.code || E2.code || E.place + " = " + E1.place + " + " + E2.place }
E → E1 - E2
    { E.place = newTemp()
      E.code = E1.code || E2.code || E.place + " = " + E1.place + " - " + E2.place }
E → E1 * E2
    { E.place = newTemp()
      E.code = E1.code || E2.code || E.place + " = " + E1.place + " * " + E2.place }
```

表达式生成三地址码的语法制导翻译

```
E → - E1
    { E.place = newTemp()
      E.code = E1.code || E.place + " = -" + E1.place }
E → ( E1 )
    { E.place = E1.place
      E.code = E1.code }
E → id
    { E.place = id.name
      E.code = "" }
```

编程实现生成表达式三地址码

① 使用递归下降分析模拟语法分析。

② 每个非终结符对应一个函数，返回表达式结果的临时变量名。

③ 使用全局变量生成临时变量名。

④ 终结符id直接返回变量名，无代码。

⑤ 生成的三地址代码通过打印输出。

完整的语法制导翻译的C语言程序代码

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX_TOKEN_LEN 64
#define MAX_CODE_LINES 1000

typedef struct {
    char code[MAX_CODE_LINES][128];
    int count;
} CodeList;

char token[MAX_TOKEN_LEN];
int pos = 0;
char input[256];

int tempCount = 0;

void getNextToken();
void error(const char* msg);
char* newTemp();
char* S();
char* E();
char* E_add();
char* E_mul();
char* E_unary();
char* E_primary();

CodeList codeList;

void addCode(const char* line) {
    if (codeList.count < MAX_CODE_LINES) {
        strcpy(codeList.code[codeList.count++], line);
    } else {
        printf("代码行数超出限制\n");
        exit(1);
    }
}
```

完整的语法制导翻译的C语言程序代码

```
41  void getNextToken() {
42      int i = 0;
43      while (input[pos] == ' ') pos++; // skip spaces
44      if (input[pos] == '\0') {
45          token[0] = '\0';
46          return;
47      }
48      if (isalpha(input[pos])) {
49          while (isalnum(input[pos])) {
50              token[i++] = input[pos++];
51          }
52          token[i] = '\0';
53      } else {
54          token[0] = input[pos++];
55          token[1] = '\0';
56      }
57  }
58
```

```
59  void error(const char* msg) {
60      printf("语法错误: %s\n", msg);
61      exit(1);
62  }
63
64  char* newTemp() {
65      static char tempName[16];
66      tempCount++;
67      sprintf(tempName, "t%d", tempCount);
68      return strdup(tempName);
69  }
70
71  int match(const char* expected) {
72      if (strcmp(token, expected) == 0) {
73          getNextToken();
74          return 1;
75      }
76      return 0;
77  }
78
```

完整的语法制导翻译的C语言程序代码

```
79  // S -> id = E
80  char* S() {
81      if (!isalpha(token[0])) error("期望标识符");
82      char idName[MAX_TOKEN_LEN];
83      strcpy(idName, token);
84      getNextToken();
85      if (!match("=")) error("期望 '='");
86      char* ePlace = E();
87      char line[128];
88      sprintf(line, "%s = %s", idName, ePlace);
89      addCode(line);
90      free(ePlace);
91      return NULL;
92  }
93
```

```
94  // E -> E + E | E - E | E * E | - E | ( E ) | id
95  char* E() {
96      return E_add();
97  }
98
99  char* E_add() {
100     char* left = E_mul();
101     while (strcmp(token, "+") == 0 || strcmp(token, "-") == 0) {
102         char op = token[0];
103         getNextToken();
104         char* right = E_mul();
105         char* temp = newTemp();
106         char line[128];
107         sprintf(line, "%s = %s %c %s", temp, left, op, right);
108         addCode(line);
109         free(left);
110         free(right);
111         left = temp;
112     }
113     return left;
114 }
115
```

完整的语法制导翻译的C语言程序代码

```
116  char* E_mul() {
117      char* left = E_unary();
118      while (strcmp(token, "*") == 0) {
119          getNextToken();
120          char* right = E_unary();
121          char* temp = newTemp();
122          char line[128];
123          sprintf(line, "%s = %s * %s", temp, left, right);
124          addCode(line);
125          free(left);
126          free(right);
127          left = temp;
128      }
129      return left;
130  }
131
```

完整的语法制导翻译的C语言程序代码

```
132  char* E_unary() {
133      if (strcmp(token, "-") == 0) {
134          getNextToken();
135          char* operand = E_unary();
136          char* temp = newTemp();
137          char line[128];
138          sprintf(line, "%s = -%s", temp, operand);
139          addCode(line);
140          free(operand);
141          return temp;
142      } else {
143          return E_primary();
144      }
145  }
146
```

```
147  char* E_primary() {
148      if (strcmp(token, "(") == 0) {
149          getNextToken();
150          char* place = E();
151          if (!match(")")) error("期望 ')'");
152          return place;
153      } else if (isalpha(token[0])) {
154          char* idName = strdup(token);
155          getNextToken();
156          return idName;
157      } else {
158          error("期望标识符或 '('");
159          return NULL;
160      }
161  }
162
```

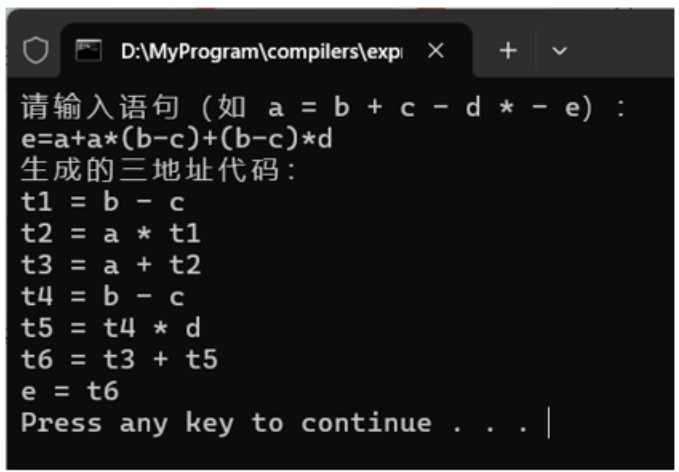
完整的语法制导翻译的C语言程序代码

```
162
163  int main() {
164      printf("请输入语句（如 a = b + c - d * - e):\n");
165      if (!fgets(input, sizeof(input), stdin)) {
166          printf("输入错误\n");
167          return 1;
168      }
169      // 去除换行符
170      input[strcspn(input, "\n")] = 0;
171
172      codeList.count = 0;
173      tempCount = 0;
174      pos = 0;
175      getNextToken();
176
177      S();
178
179      printf("生成的三地址代码: \n");
180      for (int i = 0; i < codeList.count; i++) {
181          printf("%s\n", codeList.code[i]);
182      }
183
184      return 0;
185  }
```

输入表达式：e=a+a*(b-c)+(b-c)*d ，

生成三地址代码：

t1 = b - c
t2 = a * t1
t3 = a + t2
t4 = b - c
t5 = t4 * d
t6 = t3 + t5
e = t6



常用语句的翻译

1、声明语句的翻译

- 作用:

  声明语句用于对程序中规定范围内使用的各类变量、常数、过程进行说明。

- 编译要做的工作

  ① 在符号表中建立相应的表项，填写有关的信息。如类型、嵌套深度、相对地址等。

  ② 相对地址:相对静态数据区基址或活动记录中局部数据区基址的一个偏移值。

声明语句的翻译

- 变量声明语句

① 变量的类型可以确定变量需要的内存

即类型的宽度, 可变大小的数据结构只需要考虑指针

② 函数的局部变量总是分配在连续的区间；

因此给每个变量分配一个相对于这个区间开始处的相对地址

③ 变量的类型信息保存在符号表中；

C语言变量声明的文法

P →D

D → T B ; D | ε

T → int | float | struct '{' D '}';

B → id C

C → ε | [num] C

D 生成一系列声明；

T 生成不同的类型；

B 生成简单标识符和数组标识符；

C 表示数组分量，生成[num]序列；

注意: struct是结构体类型声明。

声明序列的语法制导翻译

- 在处理一个过程/函数时，局部变量应该放到单独的符号表中去；
- 这些变量的内存布局独立
  - 相对地址从0开始；
  - 假设变量的放置和声明的顺序相同；
  - 变量offset记录当前可用的相对地址；
  - 每“分配”一个变量，offset的值增加相应的值
- addtype(id.*entry*, C.*type*,*offset*);
- 在当前符号表(位于栈顶)中创建符号表条目，记录标识符的类型，偏移量

声明变量的语法制导翻译

- offset 是定义变量的偏移地址；
- T、B和C有综合属性type, width; type为数据类型，width为变量占用内存数；
- D有综合属性width，表示声明变量占用的总内存数；
- 全局变量t和w用于将类型和宽度信息从T传递到C→ ε,相当于C的继承属性；
- struct为结构体数据类型，该类型的变量占用的内存数为结构体元素占用的内存总数；
- addtype函数声明的变量加入符号表。

P → {offset = 0;} D

D → T {t=T.*type*,w=T.*width*;}
B ;
D_1 {D.width=D.width+D_1.width;}

D →ε {D.*width*=0;}

T → int {T.*type*=*integer*;T.*width*=4;}

T → float {T.*type*=*float*;T.*width*=4;}

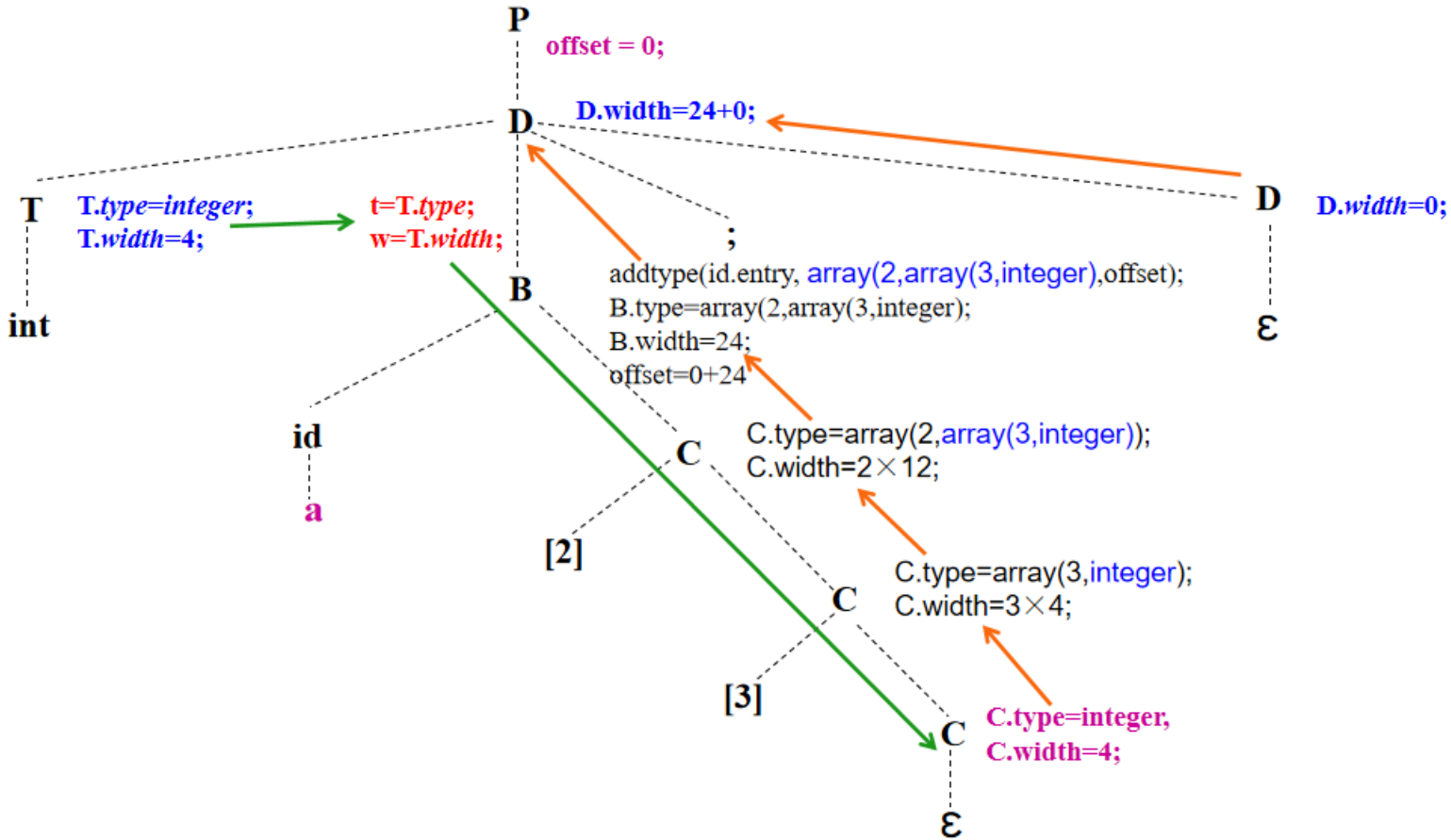
T → struct '{' D '}'; {T.*type*=struct;T.*width*=D.*width*}

B → id C {addtype(id.*entry*, C.*type*,*offset*);
B.*type*=C.*type*; B.*width*=C.*width*;
offset=*offset*+C.*width*}

C → ε {C.*type*=*t*,C.*width*=*w*;}

C →[num] C_1 { C.*type*=array(num.*value*,C_1.*type*);
C.*width*=num.*value*×C_1.*width*; }

示例：假如C语言声明二维数组：int a[2][3]

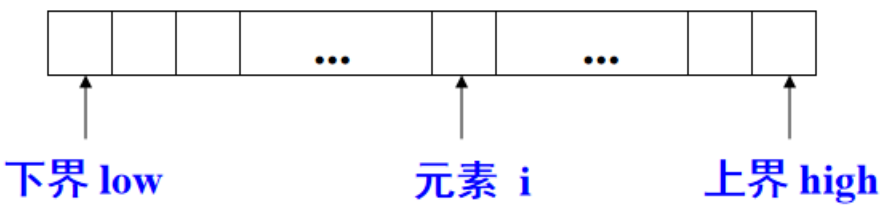


赋值语句中涉及数组元素的情况

- 数组元素地址分配

  数组元素存储在一个连续的存储块中，根据数组元素的下标查找每个元素。

- 一维数组A



基址：base

域宽：w

元素个数：high-low+1

数组元素A[i]的位置：

$$base + (i-low)\times w = i\times w + base - low\times w$$

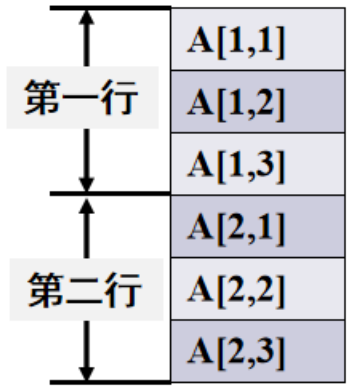
常量，在编译时可以确定

二维数组

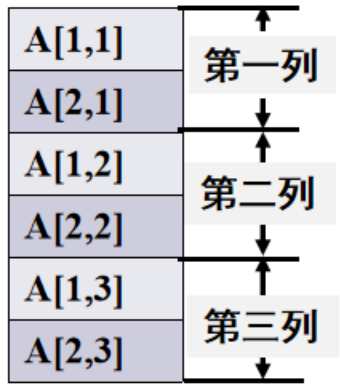
$$\begin{pmatrix} a_{0,0} & a_{0,1} & a_{0,j} & a_{0,m-1} \\ a_{1,0} & a_{1,1} & a_{1,j} & a_{1,m-1} \\ \cdots & & & \\ a_{i,0} & a_{i,1} & a_{i,j} & a_{i,m-1} \\ \cdots & & & \\ a_{n-1,0} & a_{n-1,1} & a_{n-1,j} & a_{n-1,m-1} \end{pmatrix}$$

存储方式：

按行存放
Pascal,C采用



按列存放
Fortran采用



每维的下界：$low_1=0$、$low_2=0$,每维的上界：$high_1$、$high_2$；

每维的长度：$n=high_1-low_1+1$, $m=high_2-low_2+1$； 域宽：w，基址：base

数组元素A[i, j]的位置：

$base + ((i-low_1) \times m + (j-low_2)) \times w$

$= (i \times m + j) \times w + base - (low_1 \times m + low_2) \times w$

按列存放:

$(j \times n + i) \times w + base - (low_2 \times n + low_1) \times w$

多维数组行优先存储偏移计算背景

- 假设有一个多维数组（下表编号从0开始）：

$$A[d_1][d_2][\dots][d_n]$$

访问元素： $A[i_1][i_2][\dots][i_n]$

在行优先存储中，元素的线性偏移地址（以元素大小为单位）计算公式为：

$$offset=i_1\times(d_2\times d_3\times\cdots\times d_n)+i_2\times(d_3\times\cdots\times d_n)+\cdots+i_{n-1}\times d_n+i_n$$

处理数组引用的翻译方案

$S \rightarrow$	**id** = E ;	{ $gen($ $top.get($**id**$.lexeme)$ $'='$ $E.addr)$; }
|	L = E ;	{ $gen(L.addr.base$ $'['$ $L.addr$ $']'$ $'='$ $E.addr)$; }
$E \rightarrow$	E_1 + E_2	{ $E.addr =$ **new** $Temp()$; $gen(E.addr$ $'='$ $E_1.addr$ $'+'$ $E_2.addr)$; }
|	**id**	{ $E.addr = top.get($**id**$.lexeme)$; }
|	L	{ $E.addr =$ **new** $Temp()$; $gen(E.addr$ $'='$ $L.array.base$ $'['$ $L.addr$ $']')$; }
$L \rightarrow$	**id** [E]	{ $L.array = top.get($**id**$.lexeme)$; $L.type = L.array.type.elem$; $L.addr =$ **new** $Temp()$; $gen(L.addr$ $'='$ $E.addr$ $'*'$ $L.type.width)$; }
|	L_1 [E]	{ $L.array = L_1.array$; $L.type = L_1.type.elem$; $t =$ **new** $Temp()$; $L.addr =$ **new** $Temp()$; $gen(t$ $'='$ $E.addr$ $'*'$ $L.type.width)$; } $gen(L.addr$ $'='$ $L_1.addr$ $'+'$ $t)$; }

- 为有数组引用的表达式生成三地址代码；
- 将表达式赋值给数组：S→L=E;
- 表达式含有数组因子：$E \rightarrow E_1+E_2|L$
- 数组名及下标生成：L→L[E] | id[E]

数组引用语义文法(续)

- 数组名及下标生成：L→L[E] | id[E]

- L→id[E] 生成一维数组
- L→L[E] 生成多维数组
- L.addr临时变量，用于在计算过程中累计数组引用偏移量；
- L.array是指向符号表的指针；
- L.type是L生成子数组类型；
- L.type.elem是数组元素的类型；
- E.addr存放表达式值，这里是数组元素序号；

$$
\begin{array}{lll}
L \rightarrow \textbf{id}\ [\ E\ ] & \{\ L.array = top.get(\textbf{id}.lexeme); \\
 & \quad L.type = L.array.type.elem; \\
 & \quad L.addr = \textbf{new}\ Temp\,(); \\
 & \quad gen(L.addr\ '='\ E.addr\ '*'\ L.type.width);\ \} \\
 & \\
\mid\ L_1\ [\ E\ ] & \{\ L.array = L_1.array; \\
 & \quad L.type = L_1.type.elem; \\
 & \quad t = \textbf{new}\ Temp\,(); \\
 & \quad L.addr = \textbf{new}\ Temp\,(); \\
 & \quad gen(t\ '='\ E.addr\ '*'\ L.type.width); \\
 & \quad gen(L.addr\ '='\ L_1.addr\ '+'\ t);\ \}
\end{array}
$$

数组元素作为因子

- 下面的语义文法片段是计算表达值，表达式中引用数组；
- L是代表数组的非终结符；
- L.array.base是数组基地址，被用于确定数组引用的左值；
- 使用三地址指令x=a[i]。

$E \rightarrow E_1 + E_2$		$\{\ E.addr = \textbf{new}\ Temp\,();$ $gen(E.addr\ '='\ E_1.addr\ '+'\ E_2.addr);\ \}$
|	**id**	$\{\ E.addr = top.get(\textbf{id}.lexeme);\ \}$
|	L	$\{\ E.addr = \textbf{new}\ Temp\,();$ $gen(E.addr\ '='\ L.array.base\ '['\ L.addr\ ']');\ \}$

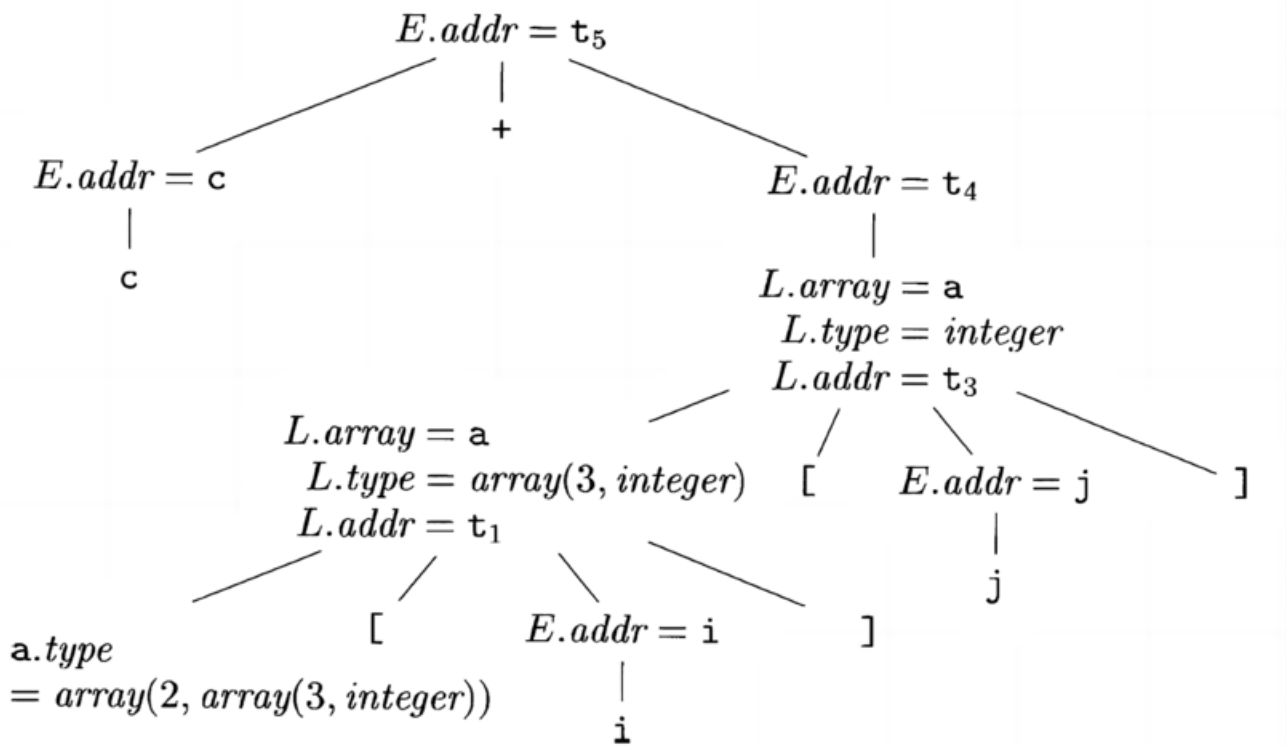
数组元素作为赋值左部

- 下面是表达式赋值的语义文法；
- 将表达式值赋给标识符：S→id = E;
- 将表达式值赋给数组：S→L=E;
- 使用三地址指令a[i]=x；

$$
\begin{array}{lll}
S \rightarrow \mathbf{id} = E\ ; & \{\ gen(\ top.get(\mathbf{id}.lexeme)\ '='\ E.addr);\ \} \\
\quad \mid\ L = E\ ; & \{\ gen(L.array\ .base\ '['\ L.addr\ ']'\ '='\ E.addr);\ \}
\end{array}
$$

带数组元素的表达式例子

- 表达式：c+a[i][j]




```
t1 = i * 12
t2 = j * 4
t3 = t1 + t2
t4 = a [ t3 ]
t5 = c + t4
```

布尔表达式和控制流语句

- 布尔表达式
  - ① 与一般的算术表达式类似
  - ② 运算符是 or、and、not等
  - ③ 运算量是布尔常量(true、false)或关系表达式(其值也是布尔量)
- 布尔表达式的作用
  - ① 计算逻辑值
  - ② 用作控制流语句中的条件表达式
- 产生布尔表达式的文法

E→ E or E
E→ E and E
E→ not E
E→ (E)
E→ id relop id
E→ true
E→ false

翻译布尔表达式的方法

- 翻译布尔表达式是为了得到表达式的运算结果
- 布尔表达式的运算结果是 true 或 false
- 如何体现运算结果？

翻译布尔表达式的方法

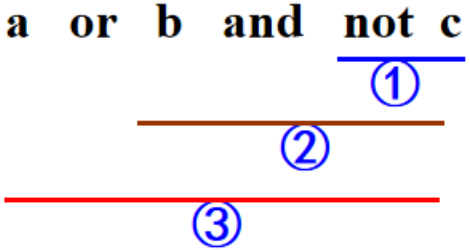
- 布尔表达式的值的表示方法
  - ① 数值表示法：
    - 1 — true　　0 — false
    - 非0 — true　0 — false
  - ② 控制流表示法：

    利用控制流到达程序中的位置来表示true或false
- 布尔表达式的翻译方法
  - ① 数值方法
  - ② 控制流方法

布尔表达式的数值翻译方法

- 把true、false数值化(true 为1，false 为0)；
- 布尔表达式的求值类似于算术表达式的求值

例如：



- 关系表达式 a<b 等价于：

if a<b
then 1
else 0

- 三地址代码

t_1:=not c
t_2:=b and t_1
t_3:=a or t_2

- a<b的三地址代码：

100: if a<b goto 103
101: t:=0
102: goto 104
103: t:=1
104:

布尔表达式的数值翻译方法

- 属性、函数及变量说明

① 属性E.place：存放布尔表达式E的值的临时变量(临时变量在符号表中的入口位置)

② 函数gen：产生并输出一条三地址语句

③ 变量nextstat：输出序列中下一条三地址语句的位置，函数gen执行之后，nextstat自动加1。

布尔表达式的数值翻译方案

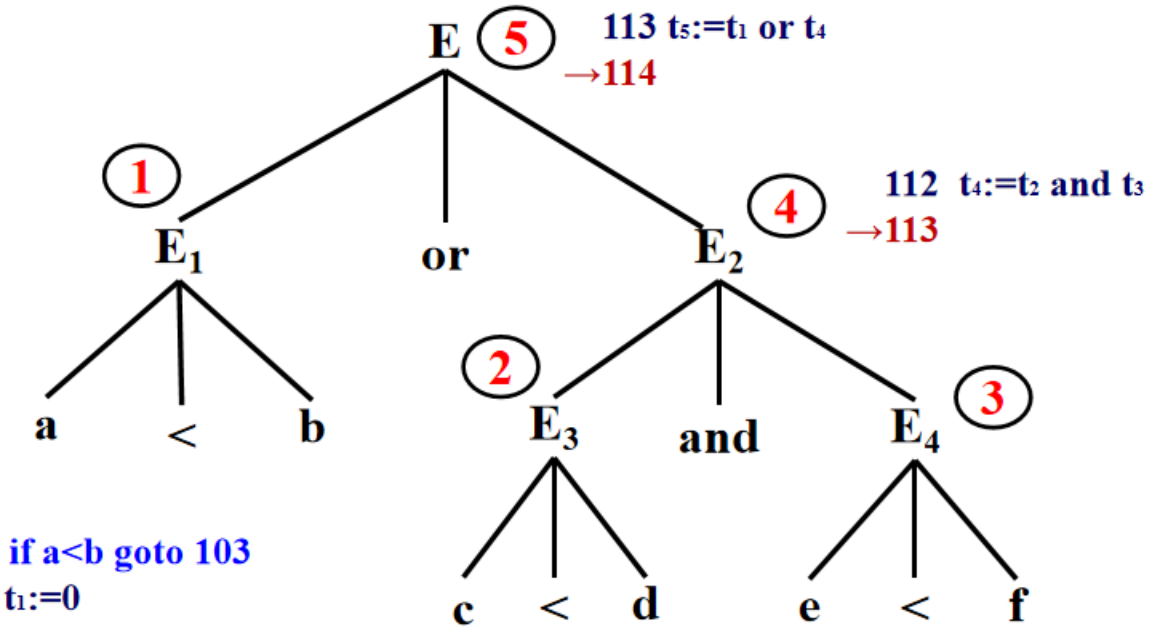
$E \rightarrow E_1$ or E_2	{ E.place:=newtemp; gen(E.place '=' E_1.place 'or' E_2.place) }
$E \rightarrow E_1$ and E_2	{ E.place:=newtemp; gen(E.place '=' E_1.place 'and' E_2.place) }
$E \rightarrow$ not E_1	{ E.place:=newtemp; gen(E.place '=' 'not' E_1.place) }
$E \rightarrow (E_1)$	{ E.place:= E_1.place) }

$E \rightarrow id_1$ relop id_2	{ E.place:=newtemp;
	gen('if' id_1.place relop.op id_2.place 'goto' nextstat+3);
	gen(E.place '=' '0');
	gen('goto' nextstat+2);
	gen(E.place '=' '1') }

$E \rightarrow$ true	{ E.place:=newtemp; gen(E.place '=' '1') }
$E \rightarrow$ false	{ E.place:=newtemp; gen(E.place '=' '0') }

示例: a<b or c<d and e<f

E→ E or E
E→ E and E
E→ not E
E→ (E)
E→ id relop id
E→ true
E→ false

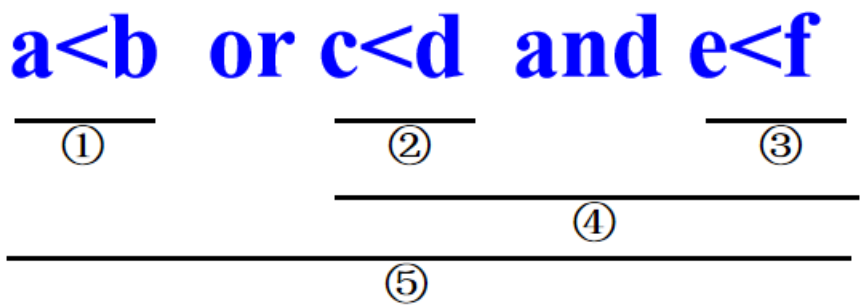


99
nextstat→100 if a<b goto 103
101 t1:=0
102 goto 104
103 t1:=1
→104

104 if c<d goto 107
105 t2:=0
106 goto 108
107 t2:=1
→108

108 if e<f goto 111
109 t3:=0
110 goto 112
111 t3:=1
→112

示例: a<b or c<d and e<f



100: if a<b goto 103
101: t_1:=0
102: goto 104
103: t_1:=1
104: if c<d goto 107
105: t_2:=0
106: goto 108
107: t_2:=1
108: if e<f goto 111
109: t_3:=0
110: goto 112
111: t_3:=1
112: t_4:=t_2 and t_3
113: t_5:=t_1 or t_4

布尔表达式的控制流翻译方法

- 把 true、false 体现在程序(三地址代码形式)的位置上

  ① 一般用此方法来实现控制流语句中的布尔表达式

  ② 如true 跳转到标号为 109 的语句，false 跳转到标号为 103 的语句

  ③ 布尔表达式的值体现在控制转移到的位置上

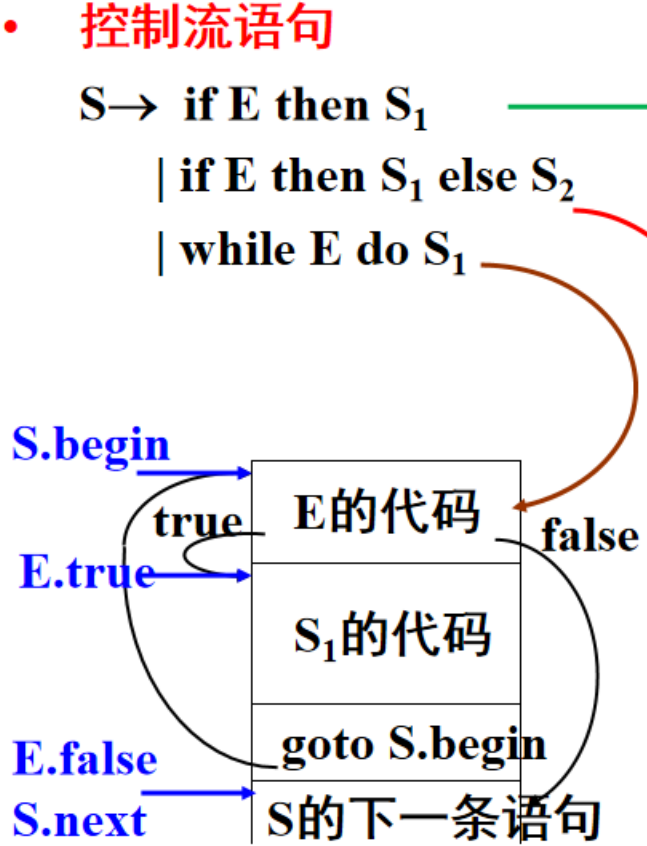
布尔表达式的控制流翻译方法

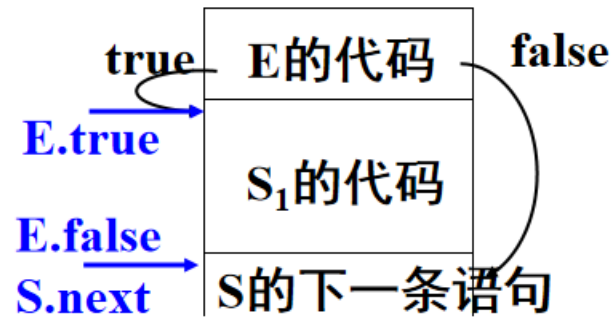
- 控制流语句

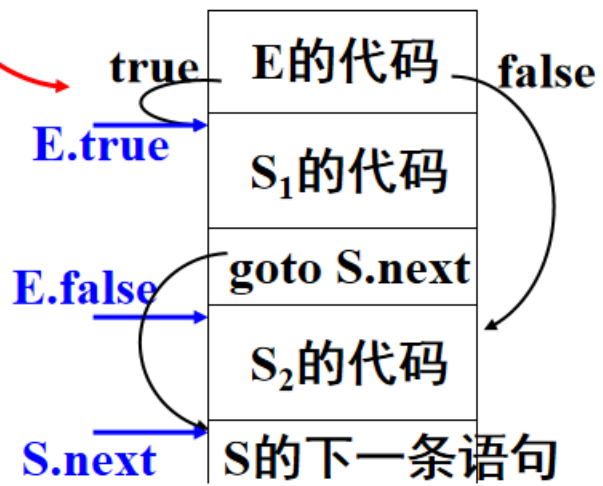
S→ if E then S_1

| if E then S_1 else S_2

| while E do S_1







布尔表达式的控制流翻译方法

- 属性说明

① 继承属性：三地址语句标号

E.true： E值为真时应执行的第一条语句的标号

E.false： E值为假时应执行的第一条语句的标号

S.next： 紧跟在语句S之后的三地址语句的标号

S.begin： 语句S的第一条三地址语句的标号

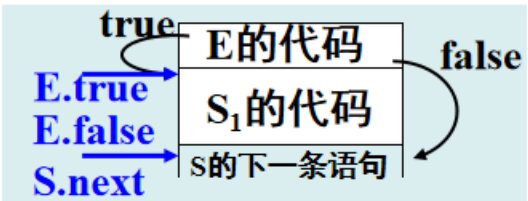
② 综合属性：三地址代码

E.code： 布尔表达式E的三地址代码

S.code： 语句S的三地址代码

翻译控制流语句的语法制导定义

S→ if E then S_1



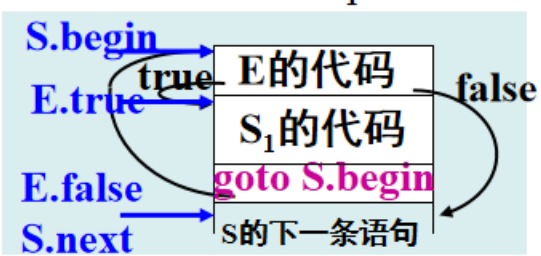
E.true =newlable; E.false=S.next; S_1.next =S.next; S.code =E.code || gen(E.true':') || S_1.code

S→ if E then S_1 else S_2



E.true =newlable; E.flase =newlable; S_1.next =S.next; S_2.next =S.next; S.code =E.code || gen(E.true':') || S_1.code || gen('goto' S.next) || gen(E.false':') || S_2.code

S→ while E do S_1



S.begin =newlable; S_1.next =S.begin; E.true =newlable; E.false =S.next; S.code =gen(S.begin':') || E.code || gen(E.true':') || S_1.code || gen('goto' S.begin)

控制流语句中布尔表达式的翻译

- 产生布尔表达式三地址代码的语法制导定义
  - ① 布尔表达式被翻译为一系列条件转移和无条件转移三地址语句
  - ② 这些语句转移到的位置是E.true、E.false之一
  - ③ 例如 a<b 翻译为：

    if a<b goto E.true

    goto E.false
- 属性说明
  - ① 继承属性

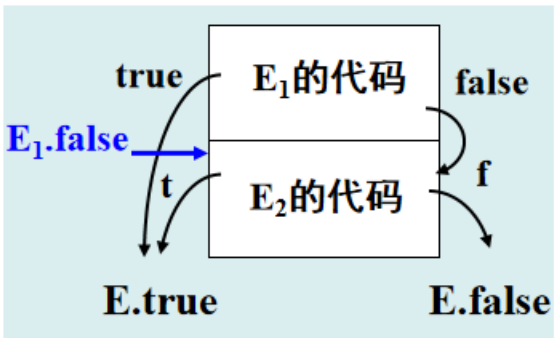
    E.true： E为真时转移到的三地址语句的标号

    E.false：E为假时转移到的三地址语句的标号
  - ② 综合属性

    E.code：为E生成的三地址代码

翻译布尔表达式的语法制导定义

$E \rightarrow E_1$ or E_2

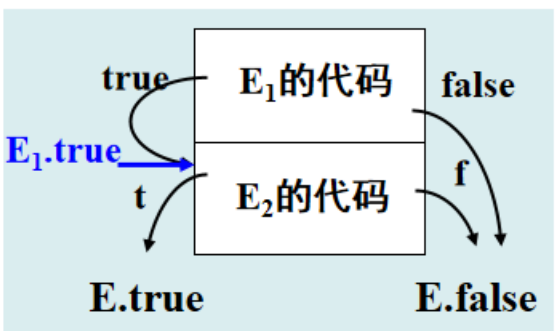


E_1.true:=E.true; E_1.false:=newlable;

E_2.true:=E.true; E_2.false:=E.false;

E.code:=E_1.code || gen(E_1.false':') || E_2.code

$E \rightarrow E_1$ and E_2



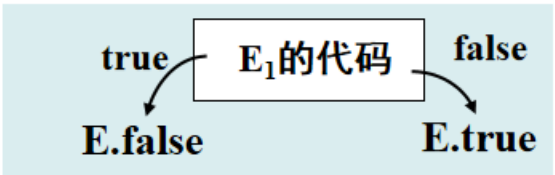
E_1.true:=newlable; E_1.false:=E.false;

E_2.true:=E.true; E_2.false:=E.false;

E.code:=E_1.code || gen(E_1.true':') || E_2.code

翻译布尔表达式的语法制导定义

$E \rightarrow$ not E_1



E_1.true:=E.false; E_1.false:=E.true

E.code:=E_1.code

$E \rightarrow id_1$ relop id_2

E.code:=gen('if' id_1.place relop.op id_2.place 'goto' E.true) || gen('goto' E.false)

$E \rightarrow (E_1)$ E_1.true:=E.true; E_1.false:=E.false;

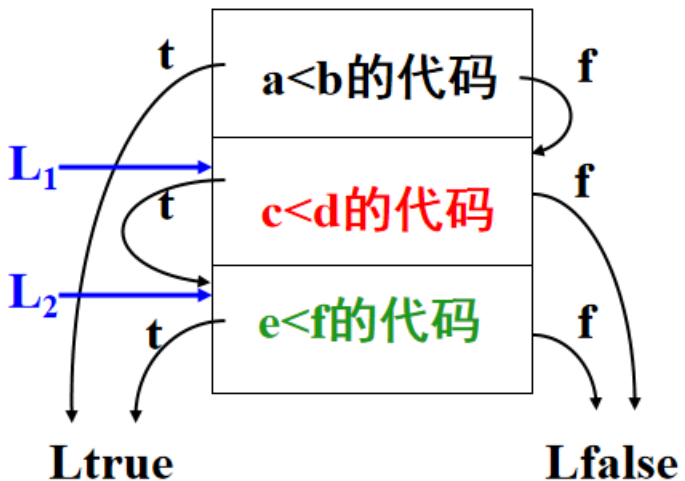
E.code:=E_1.code

$E \rightarrow$ true E.code:=gen('goto' E.true)

$E \rightarrow$ false E.code:=gen('goto' E.false)

用控制流方法翻译布尔表达式

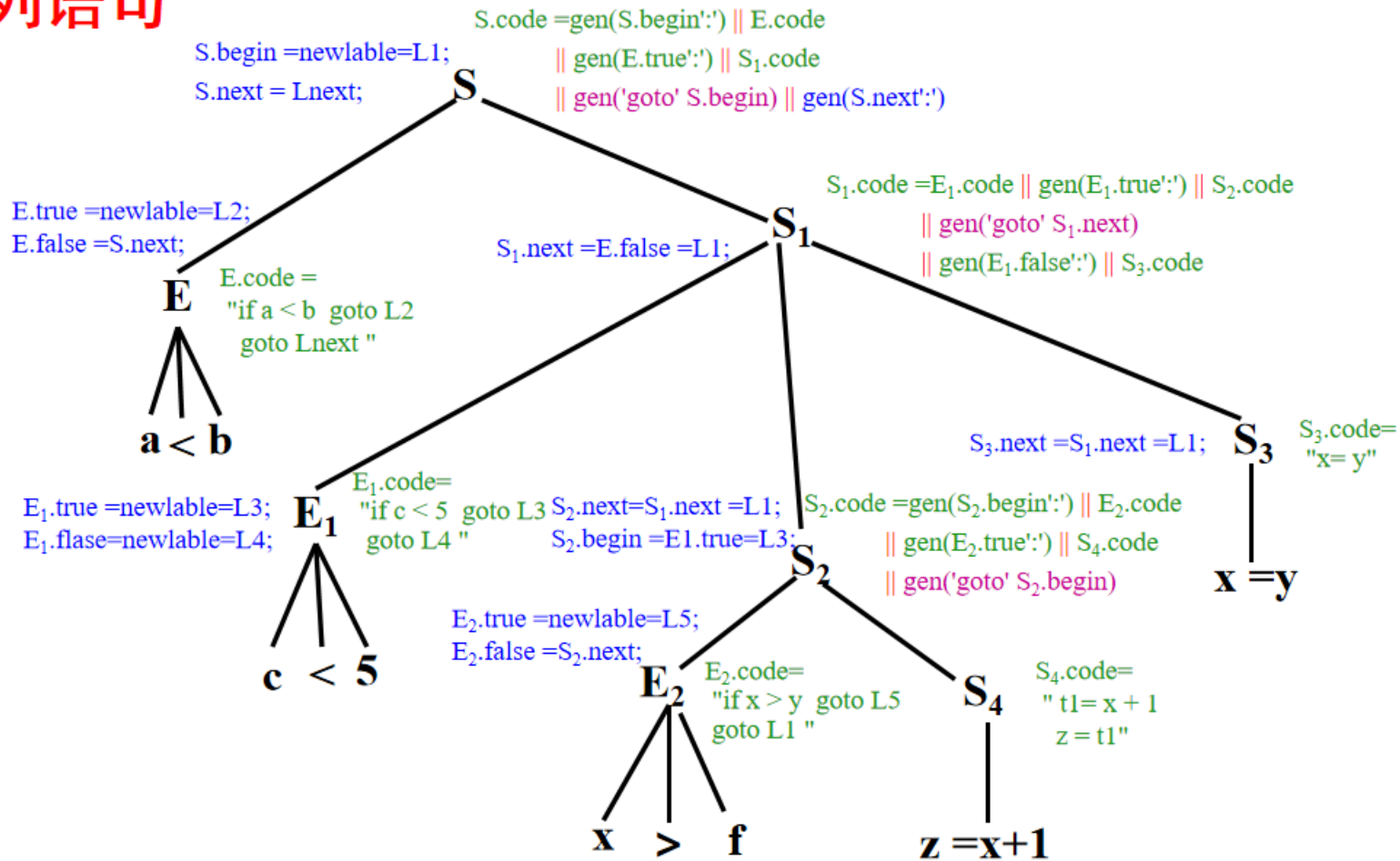
例: a<b or c<d and e<f



if a<b goto Ltrue
goto L_1
L_1: if c<d goto L_2
goto Lfalse
L_2: if e<f goto Ltrue
goto Lfalse

例：翻译下列语句

```
while a<b do
    if c<5 then
        while x>y do
            z=x+1;
    else x=y;
```



例：翻译下列语句

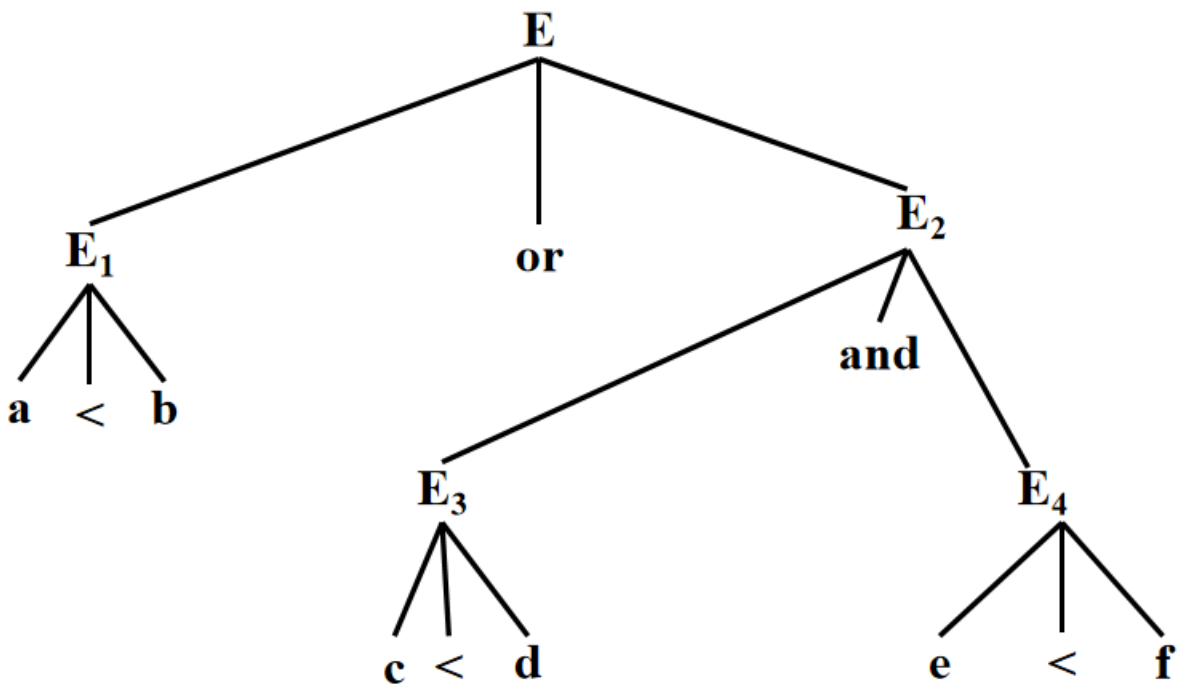
```
while a<b do
  if c<5 then
    while x>y do
      z=x+1;
  else x=y;
```

生成的三地址代码序列

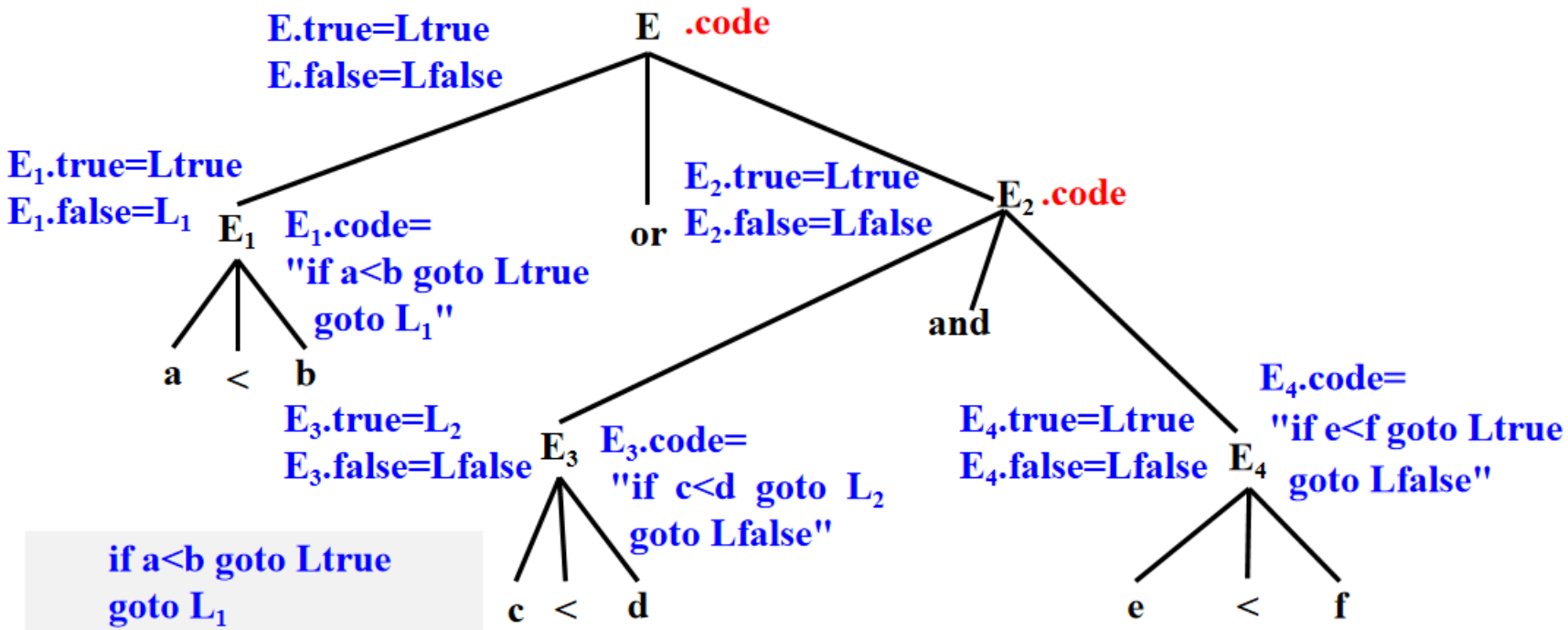
```
L1:  if a < b  goto L2
     goto Lnext
L2:  if c < 5  goto L3
     goto L4
L3:  if x > y  goto L5
     goto L1
L5:  t1 = x + 1
     z = t1
     goto L3
L4:  x = y
     goto L1
Lnext:
```

用控制流方法翻译布尔表达式

例: a<b or c<d and e<f



用控制流方法翻译布尔表达式　例: a<b or c<d and e<f



if a<b goto Ltrue
goto L_1

L1: if c<d goto L_2
goto Lfalse

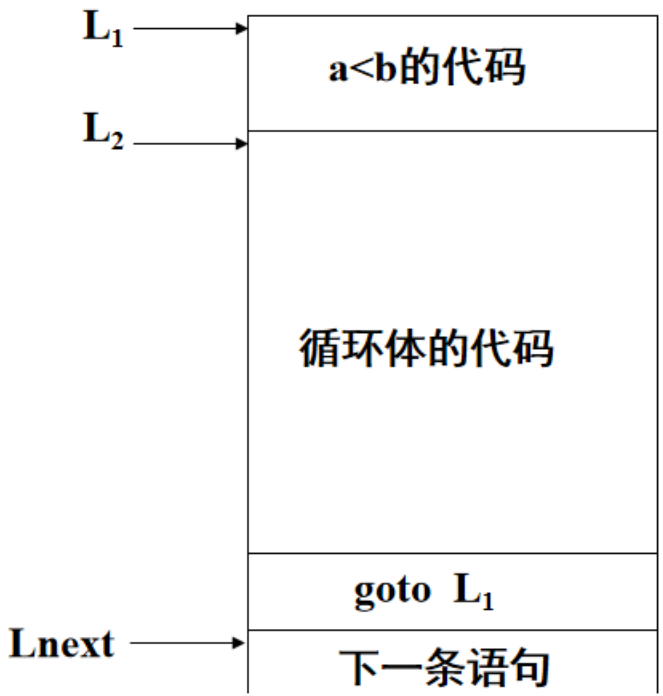
L2: if e<f goto Ltrue
goto Lfalse

两遍扫描:

1. 生成分析树
2. 翻译，在深度优先遍历分析树的过程中，计算属性值

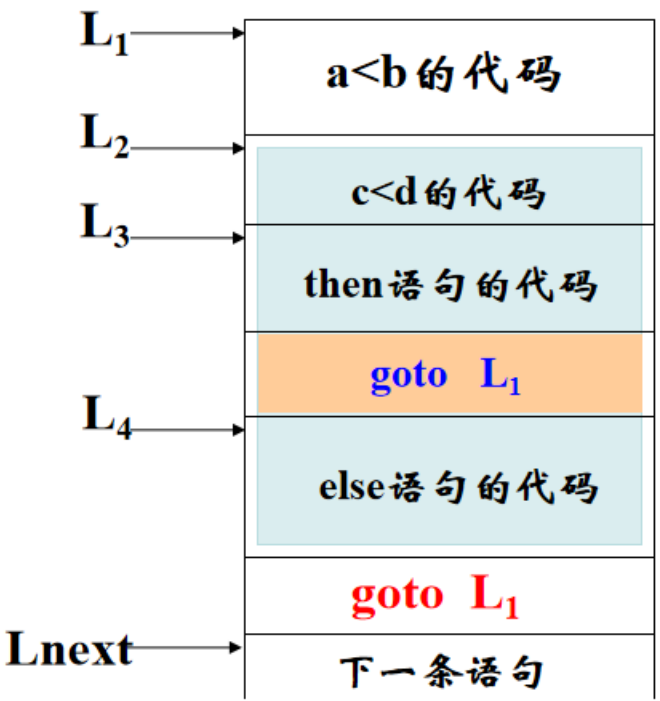
例:将下面的语句翻译为三地址代码

```
while  a<b  do
    if  c<d  then  x:=y+z
    else  x:=y-z;
```



例:将下面的语句翻译为三地址代码

```
while a<b do
    if c<d then x:=y+z
    else x:=y-z;
```



- 三地址代码为：

L_1: if a<b goto L_2
goto Lnext
L_2: if c<d goto L_3
goto L_4
L_3: t_1:=y+z
x:=t_1
goto L_1
L_4: t_2:=y-z
x:=t_2
goto L_1
Lnext:

布尔表达式代码的例子

- **if (x<100 || x > 200 && x!= y) x = 0; 的代码**

```
      if x < 100 goto L2
      goto L3
L3:   if x > 200 goto L4
      goto L1
L4:   if x != y goto L2
      goto L1
L2:   x = 0
L1:
```

回填技术

- 为布尔表达式和控制流语句生成目标代码的关键问题：某些跳转指令应该跳转到哪里
- 例如：if (E) S
  - 按照前面的翻译方法，E的代码中有一些跳转指令在E为假时执行，
  - 这些跳转指令的目标应该跳过S对应的代码。生成这些指令时，S的代码尚未生成，因此目标不确定
  - 通过语句的继承属性next来传递；需要第二趟处理。
- 如何一趟处理完毕呢？

回填技术

- 基本思想：
  - 在B的代码中，记录跳转指令位置，如：goto S.next，if … goto S.next，但是不生成跳转目标。
  - 这些位置被记录到B的综合属性B.falseList中；
  - 当S.next的值已知时（即S的代码生成完毕时），把B.nextList中的所有指令的目标都填上这个值。
- 回填技术：
  - 生成跳转指令时暂时不指定跳转目标标号，而是使用列表记录这些不完整的指令；
  - 等知道正确的目标时再填写目标标号；
  - 每个列表中的指令都指向同一个目标

利用回填技术翻译布尔表达式

- 为明确起见，生成的中间代码用四元式表示
- 四元式存放在数组中
- 用数组下标表示三地址语句的标号
- 综合属性
  - ① E.truelist：记录转移到E的真出口的指令的链表指针
  - ② E.falselist：记录转移到E的假出口的指令的链表指针
  - ③ M.instr：M所标识的三地址语句的地址
- 变量nextinstr：下一个可用的四元式地址,(产生的下一条三地址语句在数组中的位置)

利用回填技术翻译布尔表达式(辅助函数说明)

① **makelist(i)：**

a) 建立新链表，其中只包括四元式指令在数组中的下标i；

b) 返回所建链表的指针。

② **merge(p_1, p_2)：**

a) 合并由指针p_1和p_2所指向的两个链表

b) 返回结果链表的指针。

③ **backpatch(p, i)：**

用目标地址i回填 p所指链表中记录的每一条转移指令。

④ **gen(S)**

a) 产生一条三地址语句S，并写入输出数组中

b) 该函数执行完后，变量 nextinstr 加 1

利用回填技术翻译布尔表达式

- 布尔表达式的文法

  $E \rightarrow E_1$ or M E_2

  $E \rightarrow E_1$ and M E_2

  $E \rightarrow$ not E_1

  $E \rightarrow (E_1)$

  $E \rightarrow id_1$ relop id_2

  $E \rightarrow$ true

  $E \rightarrow$ false

  $M \rightarrow \varepsilon$

- 标记非终结符号M

  ① 标识布尔表达式E_2的开始位置

  ② 用属性M.instr记录其所标识的布尔表达式的第一条三地址语句的地址

  ③ 相应于产生式$M \rightarrow \varepsilon$的动作：M.instr:=nextinstr

利用回填技术翻译布尔表达式

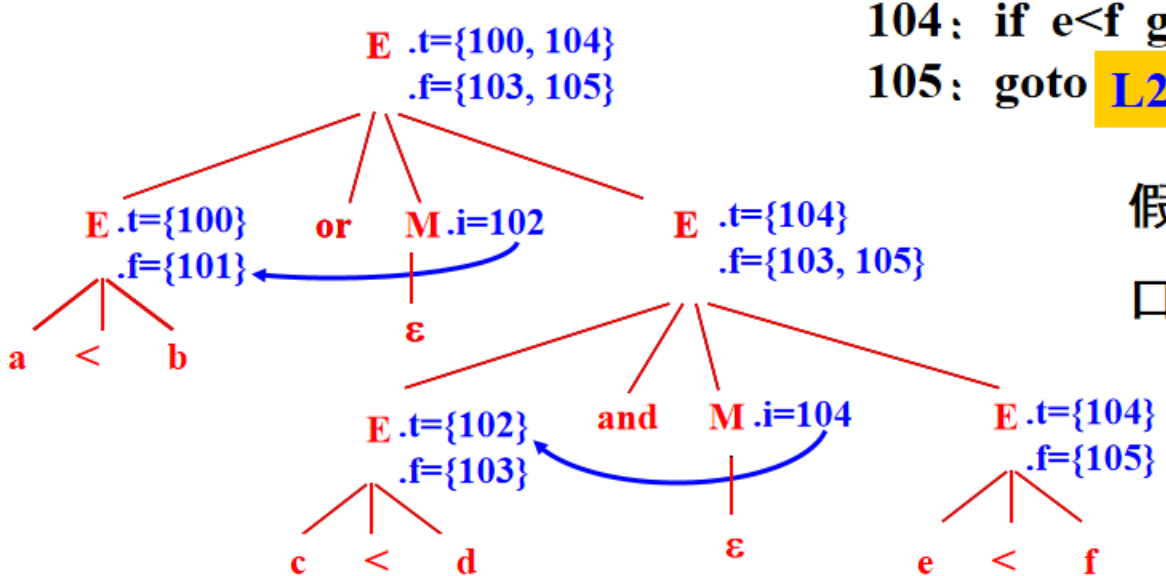
$E \rightarrow E_1$ or M E_2 { *backpatch* (E_1.*falselist* , *M.instr*) ; E.*truelist* := *merge* (E_1.*truelist*, E_2.*truelist*) ; E.*falselist* := E_2.*falselist* }	$E \rightarrow$ id_1 relop id_2 { E.*truelist* := *makelist* (*nextinstr*) ; E.*falselist* := *makelist* (*nextinstr* + 1) ; *gen* ('if' id_1.*place* relop.*op* id_2.*place* 'goto_') ; *gen* ('goto_') }
$E \rightarrow E_1$ and M E_2 { *backpatch* (E_1.*truelist* , *M.instr*) ; E.*truelist* := E_2.*truelist* ; E.*falselist* := *merge* (E_1.*falselist*, E_2.*falselist*)}	$E \rightarrow$ true { E.*truelist* = *makelist* (*nextinstr*) ; *gen* ('goto_') }
$E \rightarrow$ not E_1 { E.*truelist* := E_1.*falselist* ; E.*falselist* := E_1.*truelist* }	$E \rightarrow$ false { E.*falselist* = *makelist* (*nextinstr*) ; gen('goto_') }
$E \rightarrow$ (E_1) { E.*truelist* := E_1.*truelist* ; E.*falselist* := E_1.*falselist* }	$M \rightarrow \varepsilon$ { *M.instr* = *nextinstr* }

利用回填技术翻译布尔表达式

- a<b or c<d and e<f

假定变量nextinstr的初值为100

100：if a<b goto L1
101：goto 102
102：if c<d goto 104
103：goto L2
104：if e<f goto L1
105：goto L2



假设已知E的真假出口分别是L1和L2

利用回填技术翻译控制流语句

- 文法

$S \rightarrow$ if E then # S_1

$S \rightarrow$ if E then # S_1 & else # S_2

$S \rightarrow$ while # E do # S_1

$S \rightarrow$ begin L end

$S \rightarrow A$

$L \rightarrow L_1$; # S

$L \rightarrow S$

记录变量 nextinstr 的当前值，以便回填转移到此的指令

& 产生一条不完整的goto指令，并记录下它的位置

利用回填技术翻译控制流语句

- 改写后的文法

  S→if E then M S_1

  S→if E then M_1 S_1 N else M_2 S_2

  S→while M_1 E do M_2 S_1

  S→begin L end

  S→A

  L→L_1; M S

  L→S

  M→ε

  N→ε

属性：

E.truelist

E.falselist

M.instr

N.nextlist

S.nextlist

变量：nextinstr

函数：

makelist

backpatch

merge

gen

转移到下一条语句的指令链表的指针

M(标记非终结符)— 为了引进语义动作，产生一个标号，标识 E 或 S 的第一条四元式的位置

N(标记非终结符)— 为了引进语义动作，在 S_1 的最后产生一条转移语句跳过 S_2 的代码

利用回填技术翻译控制流语句（翻译方案）

$S \rightarrow$ if E then M_1 S_1 N else M_2 S_2 { *backpatch* (*E.truelist* , M_1*.instr*) ; *backpatch* (*E.falselist* , M_2*.instr*) ; *S.nextlist* = *merge* (S_1*.nextlist* , *merge*(*N.nextlist* , S_2*.nextlist*)) }
$N \rightarrow \varepsilon$ { *N.nextlist* = *makelist* (*nextinstr*) ; *gen* ('*goto_*') }
$M \rightarrow \varepsilon$ { *M.instr* = *nextinstr* }
$S \rightarrow$ if *E* then *M* S_1 { *backpatch* (*E.truelist* , *M.instr*) ; *S.nextlist* = *merge* (*E.falselist*, S_1*.nextlist*) }

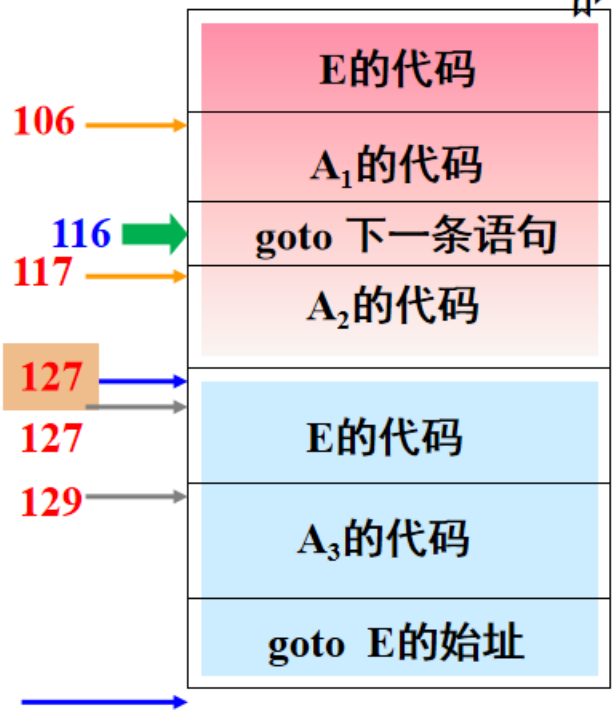
利用回填技术翻译控制流语句（翻译方案）

$S \rightarrow$ while M_1 E do M_2 S_1 { *backpatch* (S_1.*nextlist* , M_1.*instr*) ; *backpatch* (E.*truelist* , M_2.*instr*) ; S.*nextlist* := E.*falselist* ; *gen*('goto' M_1.*instr*) }
$S \rightarrow$ begin L end { S.*nextlist* := L.*nextlist* }
$S \rightarrow$ A { S.*nextlist* := nil }
$L \rightarrow L_1$; M S { *backpatch* (L_1.*nextlist* , M.*instr*) ; L.*nextlist* := S.*nextlist* }

例：

if a<b or c<d and e<f then 106 A_1 116 else 117 A_2;

127 while 127 a<b do 129 A_3




```
100：if a<b goto 106
101：goto 102
102：if c<d goto 104
103：goto 117
104：if e<f goto 106
105：goto 117
106：
…    A1的代码
115：
116：goto 127
117：
…    A2的代码
126：
127：if a<b goto 129
128：goto —
129：
…    A3的代码
138：
139：goto 127
```